

https://github.com/jambao24/VineMapper-jambao24/blob/main/projects/East_Southeast_Asian_Groups_per_county/

using the original as a template

https://github.com/winstonhoyle/VineMapper/blob/main/projects/ethnicity/East_Asian_Groups_Per_County/FormatData.ipynb

2026.01.09- downloading cb_2024_us_county_500k.zip from here <https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-and-equivalent-for-united-states-1-500000>

actually this may be more accurate? <https://www2.census.gov/geo/tiger/GENZ2024/shp/>

```
import requests
import geopandas as gpd
import pandas as pd
```

```
###Open County data
```

```
from osgeo import gdal
gdal.SetConfigOption('SHAPE_RESTORE_SHX', 'YES')
```

```
# https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-subdivision-for-united-states-1-500000
# https://stackoverflow.com/questions/61436956/set-shape-restore-shx-config-option-to-yes-to-restore-or-create-it
# requires both .shp and .shx files in file directory
file_path = "/content/cb_2024_us_county_500k.shp"
counties_gdf = gpd.read_file(file_path)
```

```
### Get Ethnic Data
```

```
r = requests.get("https://api.census.gov/data/2023/acs/acs5/groups/B02018.json")
columns_obj = r.json()
```

```
###Get columns to query and rename for later
```

```
columns = []
rename_vars = {}
variables = columns_obj["variables"]
```

```
for name, variable in list(variables.items()):
    v_split = variable["label"].split("!!")
    if len(v_split) < 3:
        continue
```

```
    if v_split[0] == "Estimate":
        label = v_split[-1]
        rename_vars[name] = label
```

```
    if (name.endswith("E") or name.endswith("M")) and v_split[-2] == "East Asian:":
        columns.append(name)
```

```
    if (name.endswith("E") or name.endswith("M")) and v_split[-2] == "Southeast Asian:":
        columns.append(name)
```

```
columns.append("GEO_ID")
columns_formatted = ",".join(columns)
```

```
response = requests.get(
    f"https://api.census.gov/data/2023/acs/acs5?get={columns_formatted}&for=county:*"
)
```

```
data = response.json()
columns = data[0]
rows = data[1:]
df = pd.DataFrame(rows, columns=columns)
```

```

estimate_cols = [col for col in df.columns if col.endswith("E")]

formtted_df = df[["GEO_ID", *estimate_cols]]
formtted_df[estimate_cols] = formtted_df[estimate_cols].astype(int)

formtted_df["most_common_ancestry_raw"] = formtted_df[estimate_cols].idxmax(axis=1)

print(formtted_df.columns)
print(formtted_df.most_common_ancestry_raw)

```

```

Index(['GEO_ID', 'B02018_017E', 'B02018_018E', 'B02018_015E', 'B02018_016E',
      'B02018_019E', 'B02018_012E', 'B02018_013E', 'B02018_014E',
      'B02018_010E', 'B02018_011E', 'B02018_005E', 'B02018_006E',
      'B02018_003E', 'B02018_004E', 'B02018_009E', 'B02018_007E',
      'B02018_008E', 'B02018_002E', 'B02018_020E',
      'most_common_ancestry_raw'],
      dtype='object')

```

```

0      B02018_012E
1      B02018_012E
2      B02018_005E
3      B02018_005E
4      B02018_012E
...
3217   B02018_017E
3218   B02018_017E
3219   B02018_017E
3220   B02018_012E
3221   B02018_017E

```

```

Name: most_common_ancestry_raw, Length: 3222, dtype: object
/tmp/ipython-input-1095341868.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-v

```
formtted_df[estimate_cols] = formtted_df[estimate_cols].astype(int)
```

```

/tmp/ipython-input-1095341868.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-v

```
formtted_df["most_common_ancestry_raw"] = formtted_df[estimate_cols].idxmax(axis=1)
```

```
# https://www.geeksforgeeks.org/pandas/pandas-combine-columns/
```

```

# edit: don't run the original version of this snippet, adding a new column and deleting 2 columns is too much work...
...

```

```
# combine "Chinese, except Taiwanese" and "Taiwanese" into "Chinese, incl. Taiwanese"
```

```
# 'B02018_002E' = "Chinese, except Taiwanese", 'B02018_008E' = "Taiwanese"
```

```
formtted_df["Ethnic Chinese"] = formtted_df["B02018_002E"] + formtted_df["B02018_008E"]
...

```

```
# wow autocomplete knew exactly what I wanted to do...
```

```
formtted_df['B02018_002E'] += formtted_df['B02018_008E']
```

```
formtted_df = formtted_df.drop(columns=['B02018_008E'])
```

```
# update "most_common_ancestry_raw" to B02018_002E "Chinese" for all instances where it is B02018_008E "Taiwanese"
```

```
formtted_df['most_common_ancestry_raw'] = formtted_df['most_common_ancestry_raw'].replace('B02018_008E', 'B02018_002E')
```

```
print(formtted_df.columns)
```

```

Index(['GEO_ID', 'B02018_017E', 'B02018_018E', 'B02018_015E', 'B02018_016E',
      'B02018_019E', 'B02018_012E', 'B02018_013E', 'B02018_014E',
      'B02018_010E', 'B02018_011E', 'B02018_005E', 'B02018_006E',
      'B02018_003E', 'B02018_004E', 'B02018_009E', 'B02018_007E',
      'B02018_002E', 'B02018_020E', 'most_common_ancestry_raw'],
      dtype='object')

```

```

/tmp/ipython-input-2919027995.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-v

```
formtted_df['B02018_002E'] += formtted_df['B02018_008E']
```

```
# swap columns https://stackoverflow.com/posts/53141546/revisions
'''
cols = list(formtted_df.columns)
a, b = cols.index('Ethnic Chinese'), cols.index('most_common_ancestry_raw')
# we want the order to be Ethnic Chinese, then Other Southeast Asian, then most_common_ancestry_raw
# current order is Other Southeast Asian, most_common_ancestry_raw, Ethnic Chinese
# left side of assignment line below
cols[b], cols[a] = cols[a], cols[b]
formtted_df = formtted_df[cols]
print(formtted_df.columns)
'''
'''

cols = list(formtted_df.columns)
c, d = cols.index('Ethnic Chinese'), cols.index('B02018_020E')
cols[d], cols[c] = cols[c], cols[d]
formtted_df = formtted_df[cols]
print(formtted_df.columns)
'''
```

```
'\n'cols = list(formtted_df.columns)\nc, d = cols.index('Ethnic Chinese'), cols.index('B02018_020E')\n'cols[d], cols[c] = cols[c], cols[d]\n'formtted_df = formtted_df[cols]\n'print(formtted_df.columns)\n'
```

```
def check_margin_error(row) -> str:
    geo_id = row["GEO_ID"]
    ethnicity_col = row["most_common_ancestry_raw"]
    val = row[ethnicity_col]

    if not val:
        return None

    moe_col = ethnicity_col.replace("E", "M")
    moe_val = int(df[df["GEO_ID"] == geo_id][moe_col])

    rmoe_val = abs(moe_val / val)
    if rmoe_val < 0.50:
        return variables[ethnicity_col]["label"].split("!!")[-1]
    else:
        return None
```

```
formtted_df["most_common_ancestry"] = formtted_df.apply(
    lambda row: check_margin_error(row), axis=1
)
```

```
/tmp/ipython-input-1230492617.py:10: FutureWarning: Calling int on a single element Series is deprecated and will raise a Type
moe_val = int(df[df["GEO_ID"] == geo_id][moe_col])
```

```
rename_vars["GEO_ID"] = "GEOIDFQ"
formtted_df = formtted_df.rename(columns=rename_vars)

#formtted_df = formtted_df.rename(columns={"B02018_002E": "Ethnic Chinese"})
formtted_df = formtted_df.rename(columns={"Chinese, except Taiwanese": "Ethnic Chinese"})
# https://sparkbyexamples.com/pandas/pandas-replace-values-based-on-condition/
formtted_df['most_common_ancestry'] = formtted_df['most_common_ancestry'].replace('Chinese, except Taiwanese', 'Ethnic Chinese')

print(formtted_df.columns)
#print(formtted_df.head)

'''

# https://stackoverflow.com/questions/48854943/how-can-i-download-a-pandas-dataframe-in-google-colab
from google.colab import files
formtted_df.to_csv('formtted_df.csv')
files.download('formtted_df.csv')
'''
```

```
Index(['GEOIDFQ', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',
      'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',
      'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',
      'Ethnic Chinese', 'Other Southeast Asian', 'most_common_ancestry_raw',
      'most_common_ancestry'],
      dtype='object')
'\n# https://stackoverflow.com/questions/48854943/how-can-i-download-a-pandas-dataframe-in-google-colab\nfrom google.colab import files\nformtted_df.to_csv('formtted_df.csv')\nfiles.download('formtted_df.csv')\n'
```

<https://www.arcgis.com/apps/mapviewer/index.html?>

url=https://geo.dot.gov/server/rest/services/Hosted/County_cb_2018_us_state_500k/FeatureServer&source=sd this is pretty cool

<https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-and-equivalent-for-united-states-1-500000>

2026.01.09 <https://pdxedu.maps.arcgis.com/apps/mapviewer/index.html> using 2019 login info to access ArcGIS and play with shp file...
runtime issue in Google Colab is with the Key in the file I have here...

```
KeyError: 'AFFGEOIDFQ'
```

just used the .dbf spreadsheet from the zip file that contains GEO_ID columns, reran that code snippet and got the following error:

```
ValueError: Cannot transform naive geometries. Please set a crs on the object first.
```

<https://stackoverflow.com/questions/64421284/geopandas-valueerror-cannot-transform-naive-geometries-please-set-a-crs-on-t>

```
###Merge Data

#print(counties_gdf)
# https://geopandas.org/en/stable/docs/user_guide/io.html
counties_gdf_xls = gpd.read_file("cb_2024_us_county_500k.dbf")
#print(counties_gdf_xls.columns)
#print(formtted_df.columns)

gdf = counties_gdf_xls.merge(formtted_df, on="GEOIDFQ", how="inner")

print("post merge:")
print(gdf.columns)
#print(gdf.head())

...

#gdf.set_crs('epsg:3857')
gdf = gdf.to_crs(9311)
...

#https://stackoverflow.com/questions/11250870/sqlite3-open-unable-to-open-database-file
#sqlite3_open_v2("data/EastSoutheast_Asian_Groups_Per_County.gpkg", &db, SQLITE_OPEN_CREATE | SQLITE_OPEN_READWRITE, NULL);

#https://gis.stackexchange.com/questions/298530/how-do-i-write-a-geopandas-dataframe-into-a-single-file-preferably-json-or-ge
gdf.to_file("output.json", driver="GeoJSON")
#gdf.to_file("data/EastSoutheast_Asian_Groups_Per_County.gpkg")

gdf.groupby("most_common_ancestry").size().reset_index(name="COUNT").sort_values(
    "COUNT", ascending=False
)

#print(gdf[['NAME', 'STATE_NAME', 'GEOIDFQ', 'most_common_ancestry']])
```

```
post merge:
Index(['STATEFP', 'COUNTYFP', 'COUNTYNS', 'GEOIDFQ', 'GEOID', 'NAME',
      'NAMELSAD', 'STUSPS', 'STATE_NAME', 'LSAD', 'ALAND', 'AWATER',
      'geometry', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',
      'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',
      'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',
      'Ethnic Chinese', 'Other Southeast Asian', 'most_common_ancestry_raw',
      'most_common_ancestry'],
      dtype='object')
/usr/local/lib/python3.12/dist-packages/pyogrio/geopandas.py:917: UserWarning: 'crs' was not provided. The output dataset will
write(
```

	most_common_ancestry	COUNT
3	Filipino	491
2	Ethnic Chinese	316
9	Vietnamese	62
6	Korean	53
4	Hmong	42
5	Japanese	28
0	Burmese	20
7	Laotian	13
1	Mongolian	4

ArcGIS mapping steps I used

create new map upload .zip file as base layer upload output.json (GeoJSON file) as 2nd layer (of data)

to change visibility of layers (e.g. changing color intensity, making sure county lines are visible in black)- I clicked on the layer to select it, then the Properties button in the tab on the right (top button), and clicked through the Appearance drop through until I was able to change it to my liking.

2026.01.13 map edits for better visibility after sharing-

<https://pdxedu.maps.arcgis.com/apps/mapviewer/index.html?webmap=a9ffa6184e8e44748fb2fc422e7ed25b>

There's are 2 basemaps available in the ArcGIS map by default- "World Topographic Map" and "World Hillshade". Made both completely transparent to get rid of the green from parks/forests and city labels that detract from the map

cb_2024_us_county_500k layer (feature layer)- set Appearance > Blending to "Normal" and Transparency to 75%

ACS_ESEA_data_output (from GeoJSON)- set Appearance > Blending to "Darken" and Transparency to 25%

PNG files were generated using the ArcGIS Print setting using the following parameters: A4 landscape, PNG32 format, 100 DPI

Start coding or generate with AI.